

Dynamical systems time series classification

Kirill Semkin^{1*} and Vadim Strijov^{2*}

^{1*}Moscow Russia.

^{2*}Moscow Russia.

*Corresponding author(s). E-mail(s): semkin.ki32@gmail.com;
strijov@forecsys.ru;

Keywords: time series; classification; dynamical systems; NeuralODE; inertial measurement unit

We propose a method for multivariate time series classification under the assumption that series of different classes are generated by distinct dynamical systems, modeled as ordinary or stochastic differential equations whose noisy phase trajectories form the observed data. The classifier assigns each series to the class whose dynamics best fit it, and we prove it is asymptotically optimal under suitable noise conditions. Since exact trajectories are rarely available, we replace them with conditional expectations obtained by solving a smoothing problem, and learn unknown vector fields from data via Neural Ordinary Differential Equations. The method is validated on synthetic linear systems and human activity recognition from inertial sensors.

1 Introduction

The paper introduces a method for time series classification. The *time series* is a finite sequence of scalar values $x(t_i) = \{x_{t_1}, \dots, x_{t_n}\}$ with variable length defined at

particular times $t_1, \dots, t_n \in \mathbb{R}$. The *multivariate* time series is a sequence of vector values $\mathbf{x}(t_i) = \{\mathbf{x}_{t_1}, \dots, \mathbf{x}_{t_n}\}$ where $\mathbb{R}^d$. This is equal to having d time series with the same times. Everywhere further we assume time series to be multivariate in general. From theoretical point of view it is convenient to assume time series values $\mathbf{x}_{t_i}$ to be *random vectors*. Time series have an associated *class label* $y \in 1 \dots K$ that is also random in general. Here K is the number of classes. A *classifier* is a map $\delta(\mathbf{x}(t_i))$ that associates any time series with a class label. We will seek to obtain a "good" classifier in some sense defined later.

There are many methods for time series classification in the literature. *Distance-based* approaches rely on measuring similarity between time series using distance metrics. The metric can be defined using Dynamic Time Warping [1] and the k-Nearest-Neighbors (k-NN) classifier can be used then [2]. *Feature-based* approaches extract finite set of statistical [3, 4] or spectral [5] features from time series for further classification. *Deep learning* approaches uses neural networks to automatically learn time series features from raw train data. Convolutional neural networks [6, 7], recurrent neural networks (RNN) [8] or their combination [9–11] are designed to process sequential data like time series.

In the paper we assume particular model behind time series - *dynamical system*. By that we mean the time-invariant *ordinary differential equation (ODE)* of the form

$$\begin{cases} \frac{d}{dt} \bar{\mathbf{x}}(t) = f(\bar{\mathbf{x}}), \\ \bar{\mathbf{x}}(0) = \bar{\mathbf{x}}_0, \end{cases} \quad (1)$$

where $\bar{\mathbf{x}} \in M$, $f(\mathbf{x}) : M \rightarrow M$ is a *vector field*, $\bar{\mathbf{x}}_0$ is a *starting point*, M is a differentiable manifold in general. We consider vector fields are not arbitrary but belong to some normed space $f \in (\mathcal{F}(M), \|\cdot\|)$. To find a solution $\bar{\mathbf{x}}(t)$ satisfying (1) on some time interval $t \in [0, T]$ is to solve an *initial value problem (IVP)*. We also denote IVP

as $\bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, f)$ and call it *phase trajectory*. Under some regularity conditions on f the solution is unique [12]. We associate K different vector fields f_j , $j \in 1, \dots, K$ with each series class. The series values are the noisy phase trajectories

$$\mathbf{x}(t_i) = \bar{\mathbf{x}}(t_i) + \boldsymbol{\epsilon}(t_i), \quad (2)$$

where $\boldsymbol{\epsilon}(t_i)$ is a random noise process. Its values $\boldsymbol{\epsilon}_{t_i}$ are i.i.d.random vectors (mutually for all classes and any times) having

$$\mathbb{E}[\boldsymbol{\epsilon}_{t_i}] = 0, \text{ Cov}(\boldsymbol{\epsilon}_{t_i}) = R \text{ where } \|R\|_2 \leq \sigma^2 < \infty. \quad (3)$$

Here R is covariance matrix, $\|R\|_2$ is matrix spectral norm. Therefore $\mathbb{E}[\mathbf{x}(t_i)] = \bar{\mathbf{x}}(t_i)$ and $\text{Cov}[\mathbf{x}(t_i)] = R$. In general, the classifier must account for both the deterministic component $\bar{\mathbf{x}}(t_i)$ and the noise component $\boldsymbol{\epsilon}(t_i)$ of the series. We will later show that under some conditions there is an asymptotically accurate classifier that would not rely on the noise. Then we will discard these condition and will introduce more general classifier. Moreover, we will be able to adapt the classifier to the more general dynamical system described by the *stochastic differential equation (SDE)* [13] of the form

$$\begin{cases} d\bar{\mathbf{x}}(t) = f(\bar{\mathbf{x}})dt + GdW(t), \\ \bar{\mathbf{x}}(0) = \bar{\mathbf{x}}_0, \end{cases} \quad (4)$$

where $W(t)$ is a m -dimensional Brownian motion, G is a $d \times m$ constant diffusion matrix, unique for each class. Phase trajectories $\bar{\mathbf{x}}(t)$ are *random processes* in this model. The definition of the IVP translates straightforwardly for the SDE. The IVP solution is unique under some regularity conditions on f [13].

Our approach can be extended for the less direct connection between observed time series and the phase trajectories (2). Imagine the connection is given by some

observation map $\alpha : M \rightarrow \mathbb{R}^d$ such that

$$\mathbf{x}(t_i) = \alpha(\bar{\mathbf{x}}(t_i)) + \epsilon(t_i), \quad (5)$$

where M has dimension $k \neq d$ in general. We would like to reconstruct the $\bar{\mathbf{x}}(t_i)$ from the $\mathbf{x}(t_i)$ to build a classifier. The reconstruction will be based on building an *embedding* of the manifold M . In general, an embedding is a smooth map $\phi : M \rightarrow N$ into another manifold N that is a diffeomorphism onto its image $\phi(M)$. It follows that phase trajectories $\bar{\mathbf{x}}(t) \subset M$ and $\phi(\bar{\mathbf{x}}(t)) \subset \phi(M)$ are basically identical (more strictly, diffeomorphic). To reconstruct the phase trajectories we need to build $\phi(\bar{\mathbf{x}}(t_i))$ from the observed time series $\mathbf{x}(t_i)$. One way to archive this is the Whitney Embedding Theorem [14]. If α is a generic map [?] from M to $\mathbb{R}^{2k+1}$ then $\alpha = \phi$ and therefore $\mathbf{x}(t_i) = \phi(\bar{\mathbf{x}}(t_i))$. This is basically the original connection (2) but with the diffeomorphic phase trajectories instead of the original one. However, this approach requires an excessive $2k + 1$ number of time series even when k is rather small. Another approach is the Takens embedding theorem [15]. It requires only one observed scalar time series $x(t_i)$. If α is a generic map from M to $\mathbb{R}$ then the embedding is given by the map

$$\phi(\bar{\mathbf{x}}) = (\alpha(x), f(\alpha(x)), \dots, f^m(\alpha(x))),$$

where $m > 2k$. Substituting $\bar{\mathbf{x}}(t_i)$ gives us $\phi(\bar{\mathbf{x}}(t_i)) = (x(t_i), x(t_{i-1}), \dots, x(t_{i-k}))$ which we call *delay vector*. Illustration of the Takens theorem is on the Fig. 1. The delay vectors form noisy diffeomorphic phase trajectory because $x(t_i)$ is itself noisy. So they are basically identical to $\mathbf{x}(t_i)$ from (2). Further, we would not differentiate reconstructed phase trajectories and the original one.

Our time series' model can be applied to many real-word classification problems. For example, in medicine an electroencephalogram (EEG) is a direct, macroscopic

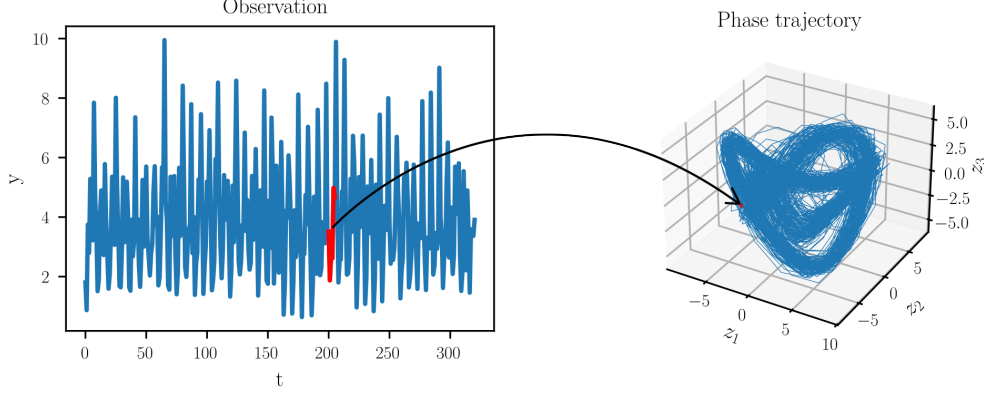


Figure 1 Application of the Takens theorem. On the left are observations $x(t_i)$. To build a point of the phase trajectory $\tilde{\mathbf{x}}(t_i)$ we must take several lagged observation values (colored in red), e.g. $\tilde{\mathbf{x}}(t_i) = (x(t_i), x(t_{i-1}), x(t_{i-2}))^n$.

measure of the brain’s electrical activity [16]. The activity is governed by the Maxwell’s Equations that can be reduced to the ODEs [17]. The EEG can be used for detecting arrhythmias or other heart conditions [6] as well as to identify seizures and sleep stages from brainwave data [7]. In the weather forecasting the fundamental equations of atmospheric dynamics can also be reduced to an ODEs system [18]. The classification problems in this field include weather regime classification [19] and climate pattern identification [20]. In human activity recognition (HAR) ODEs are primarily used to model the dynamics of human motion. The core idea is that human movement is a continuous physical process governed by laws of mechanics, which can be described by differential equations [21]. This idea can be applied to the basic or fine-grained activity recognition [22, 23], fall detection [24], and other tasks. To conclude, our model is applicable when it is reasonable to assume that observed time series come from some dynamical system of the form (1) or (4).

Our classifier will rely on the knowledge of the vector fields f_j which is not always available. To compensate this we will build approximate fields from data using Neural Ordinary Differential Equations (NODE) [25]. It allows to use gradient optimization when an optimized objective depends on ODE trajectories. There are plenty of papers

using NODE for regression tasks. For example, NODE are used in the context of the physics-informed ODE [26, 27] and control learning [28, 29]. However, very few papers analyze NODE in the context of time series classification. Authors of [30] reinterpreted convolutional neural networks in terms of latent ODE dynamics and used it for hyperspectral image classification. Paper [31] solved classification problem in terms of control theory and NODE. Our paper contributes to the NODE applications for the general dynamical systems.

The NODE paper [25] introduced its own popular time series model called latent ODE. It is illustrated on the Fig. 2. The model is probabilistic and in our terms it can be described as

$$\begin{aligned}\bar{\mathbf{x}}_{t_0} &\sim p_0(\bar{\mathbf{x}}_{t_0}), \\ \bar{\mathbf{x}}_{t_1}, \bar{\mathbf{x}}_{t_2}, \dots, \bar{\mathbf{x}}_{t_N} &= \text{ODESolve}(\bar{\mathbf{x}}_{t_0}, f, t_0, \dots, t_N), \\ \text{each } \mathbf{x}_{t_i} &\sim p_x(\mathbf{x}_{t_i} | \bar{\mathbf{x}}_{t_i})\end{aligned}$$

where $\text{ODESolve}()$ is an approximate solution of the initial value problem using ODE solver [32]. Authors used a variational autoencoder (VAE) [33] to inference the distributions p_0, p_x and the vector field f from the $\mathbf{x}_{t_i}$. While this model is more general and is widely used [34–36], it has several drawbacks compared to ours. First, it has to parameterize f, p_x and the VAE. Hence, more data is needed in general to obtain good approximations of these functions. Our approach may only need to parameterize f . Second, in the latent ODE model a connection between the phase trajectories and the observed time series is given by the general distribution p_x . Thus, the connection is difficult to interpret. In addition, the VAE is needed to reconstruct the $\bar{\mathbf{x}}(t_i)$ from the $\mathbf{x}(t_i)$. Our reconstruction approaches discussed above give explicit and easy ways to obtain $\bar{\mathbf{x}}(t_i)$ from the $\mathbf{x}(t_i)$ and vice versa. Actually, several papers can be found in

the literature [37, 38] that use Takens theorem ideas with the NODE to restore latent dynamics. But unfortunately, it is poorly covered for machine learning applications. Our paper aims to close this gap.

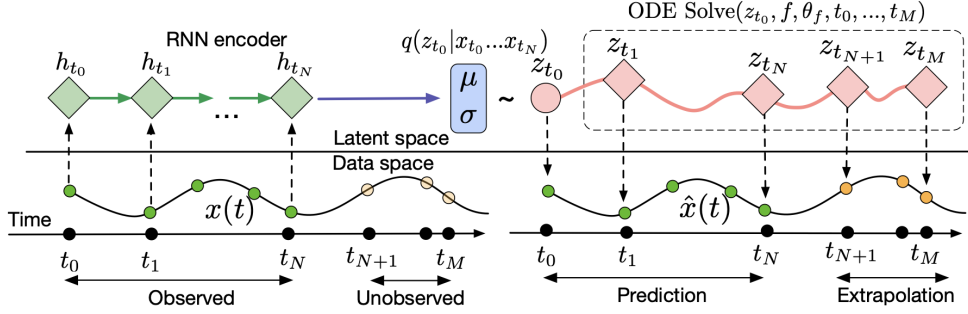


Figure 2 Computational graph of the latent ODE model. Source: reprinted from Chen et al.(2018). Here $z(t_i)$ correspond to the $\tilde{\mathbf{x}}(t_i)$.

In the next section we will describe a general principal for the time series classification inside our model. We will build an asymptotically optimal classifier under mentioned properties of the noise ϵ_{t_i} and several additional conditions. These conditions will help with a theoretical prove but may not hold in practice. So we will progressively discard this conditions and adapt classifier for a more general settings. We will also show how adapt the classifier to the SDE case (4). Finally, we will describe the procedure of building vector field approximations from the data. Then we will apply our method to three classification tasks for emprirical validation. The first will be the classification of three linear ODE systems. The second will be classification of the human motions from the Inertial Measurement Unit (IMU) sensors. The last one will be classification of heart condition from the ECG recordings.

2 Asymptotically optimal classifier

The main principle for building a classifier in the paper is the proximity of an observed time series $\mathbf{x}(t_i)$ to the theoretical initial value problem solution $\bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, f_j)$ for each class $j \in 1 \dots K$. Basically, our classifier is distance-based.

Denote by $\rho_n(\mathbf{x}(t_i), \mathbf{y}(t_i))$ the metric between time series $\mathbf{x}(t_i)$, $\mathbf{y}(t_i)$ of length n defined as

$$\rho_n(\mathbf{x}(t_i), \mathbf{y}(t_i)) = \sum_{i=1}^n \|\mathbf{x}(t_i) - \mathbf{y}(t_i)\|_2^2. \quad (6)$$

Then, define the classifier as

$$\delta(\mathbf{x}(t_i)) = \arg \min_{j \in 1 \dots K} \rho_n(\mathbf{x}(t_i), \bar{\mathbf{x}}(t_i|\bar{\mathbf{x}}_0, f_j)). \quad (7)$$

It turns out that this theoretical classifier has vanishing probability of misclassification as the length of the time series increases. But starting point $\bar{\mathbf{x}}_0$ must be known exactly and phase trajectories from different systems must diverge in some sense. The formal result:

Theorem 1. *Let $\mathbf{x}(t_i) = \{\mathbf{x}_{t_1}, \dots, \mathbf{x}_{t_n}\}$ be the time series to classify defined for all $n \in \mathbb{N}$. Let $\bar{\mathbf{x}}_0$ be the starting point of the associated phase trajectory $\bar{\mathbf{x}}(t_i)$. And let*

$$\rho_\infty(\bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, f_j), \bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, f_p)) = +\infty \quad \forall j \neq p; \quad j, p \in 1 \dots K.$$

Denote by y the true class of the time series. Then, under proposed conditions on the noise (3), the following holds

$$\mathbb{P}(\delta(\mathbf{x}(t_i)) \neq y) \rightarrow 0, \quad n \rightarrow \infty.$$

The proof of the theorem is deferred to [Appendix A](#).

3 Why optimal classifier may be intractable

Unfortunately, the classifier (7) relies on precise IVP solutions and Theorem 1 relies on precise starting point $\bar{\mathbf{x}}_0$. These requirements are often too strong on practice. We now provide several reasons for that and propose a new classifier.

3.1 Approximate vector fields

Imagine instead of the true vector fields f_j we only have approximate versions $\tilde{f}_j$ such that

$$\|f_j - \tilde{f}_j\| \leq r_j,$$

where $f_j, \tilde{f}_j \in \mathcal{F}(M)$ and r_j are approximation error bounds. This situation always arises when one approximates vector fields from the data (see Section 6).

In general, any small approximation error is enough for phase trajectories from f and $\tilde{f}$ to diverge. We show an Example 1 of such system in Appendix A. The visualization of the associate trajectories is on the Fig. 3 That makes classifier (7) useless. Nonetheless, for a large family of vector fields the $\bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, \tilde{f}_j)$ are arbitrary close to $\bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, f_j)$ if r_j is sufficiently small and times are bounded $t < T$. This is called Grönwall-type bounds and can be found in ODE literature [39]. So further we assume approximation errors are small enough and vector fields f_j are considerably different to not differentiate between $\bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, \tilde{f}_j)$ and $\bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, f_j)$. We also assume that vector fields f_j are considerably different across classes to exclude situations like in Example 1.

3.2 Approximate IVP solutions and noisy phase trajectories

Even if vector fields are known exactly, the IVP is almost always intractable [32]. The numerical methods called ODE solvers are able to approximate IVP solution with arbitrary precision [32] for regular vector fields. The main approximation parameter

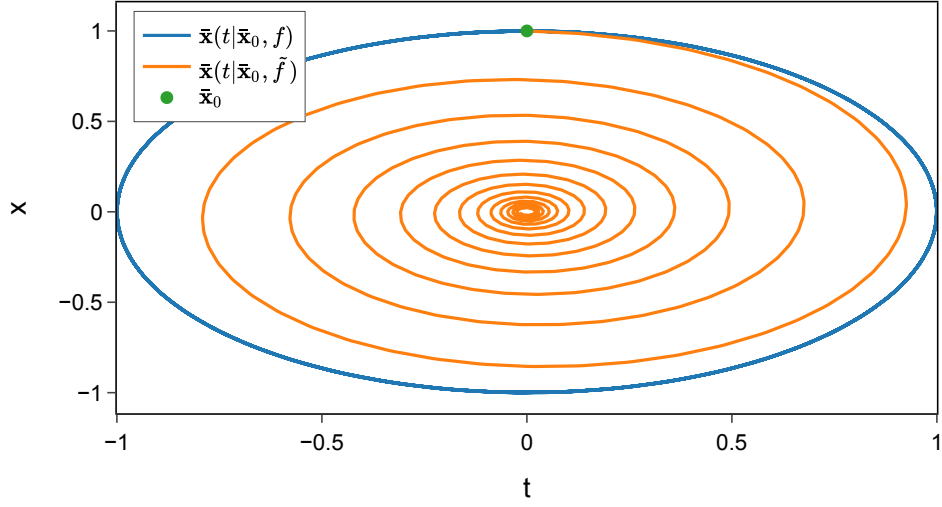


Figure 3 Diverging phase trajectories of the close vector fields $\|f - \tilde{f}\| \leq 0.1$ with $\bar{\mathbf{x}}_0 = (0, 1)^\top$. While $\bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, f)$ is a circle trajectory (called center in dynamical systems), the $\bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, \tilde{f})$ is a spiral sink that diverges to the $\mathbf{0}$.

of the solvers is an integration step τ . A solver has order of convergence l if

$$\sup_{t \in [0, T]} |\text{ODESolve}(t|\bar{\mathbf{x}}_{t_0}, f) - \bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, f)| = \mathcal{O}(T\tau^l). \quad (8)$$

The higher the convergence order and the smaller τ the higher computational complexity of the ODESolve procedure. Besides this tradeoff some vector fields imply additional constraints on the smallness of the τ . If not satisfied, solver solution diverges from the true one. This effect is called *stiffness* and is usually addressed by choosing an appropriate solver [40]. The illustration of this effect is on the Fig. 4.

As follows from (8) applying ODESolve to find full trajectory $t \in [0, T]$ starting from the exact $\bar{\mathbf{x}}_0$ becomes impractical for long trajectories. We also assume

$$\bar{\mathbf{x}}_0 \sim F(\cdot) \quad (9)$$

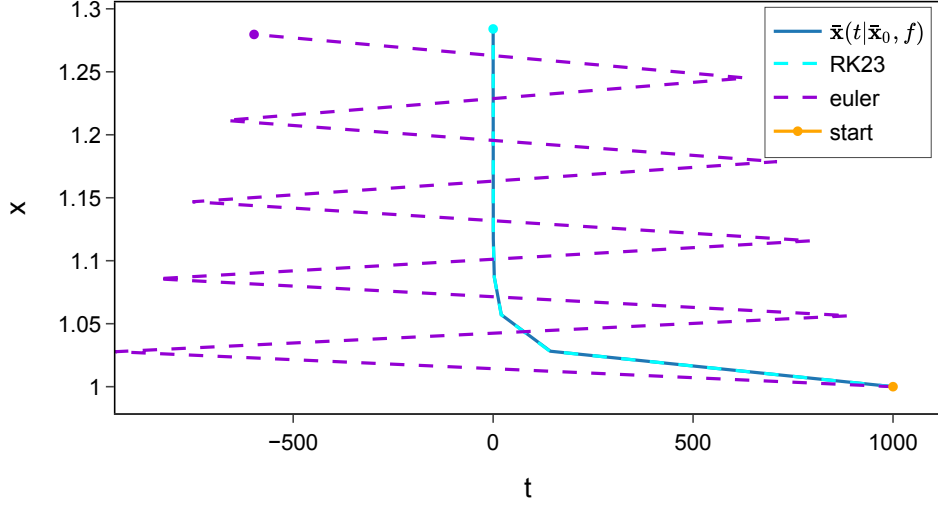


Figure 4 Exact and approximate IVP solutions for the stiff system $f(\bar{\mathbf{x}}) = \begin{pmatrix} -35 & 0 \\ 0 & 0.5 \end{pmatrix} \bar{\mathbf{x}}$ and $\bar{\mathbf{x}}_0 = (1000, 1)^T$. The Runge-Kutta23 (RK23) ODE solver approximates exact solution with high precision. The Euler (euler) ODE solver is unstable and diverges from the exact solution. Integration step $\tau = 0.05$ is equal for both solvers but for Euler it is not small enough. The RK23 has higher convergence order and is stable under such τ .

to be a random vector in general where $F(\cdot)$ is its distribution function. Hence, the classifier 7 can not be applied in these conditions.

On the other hand, imagine we knew $\bar{\mathbf{x}}_0$ and $\bar{\mathbf{x}}_{t_i}$ for some $i \in 1 \dots n$. We could then approximate $\bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, f), t \in [0, t_i]$ using $\text{ODESolve}(t|\bar{\mathbf{x}}_{t_0}, f)$, and approximate $\bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, f), t \in [t_i, T]$ using $\text{ODESolve}(t|\bar{\mathbf{x}}_{t_i}, f)$. The solver's accuracy increases. To obtain practical solution, we could simply replace $\bar{\mathbf{x}}_{t_i}$ with $\mathbf{x}_{t_i}$. But if the σ from (3) is large enough then the starting points differ considerably with high probability. This leads to very different IVP solutions $\text{ODESolve}(t|\mathbf{x}_{t_i}, f), \text{ODESolve}(t|\bar{\mathbf{x}}_{t_i}, f)$ in general [39]. We must take randomness into account in a different way.

4 Classification as smoothing problem

Let's break dynamics (1) into consecutive time segments $[t_0, t_1), [t_1, t_2), \dots, [t_{n-1}, t_n]$ so we have n IVPs

$$\begin{cases} \frac{d}{dt}\bar{\mathbf{x}}(t) = f(\bar{\mathbf{x}}), \\ \bar{\mathbf{x}}(0) = \bar{\mathbf{x}}_{t_i}, \\ t \in [0, t_{i+1} - t_i]. \end{cases} \quad (10)$$

Let's approximate each IVP solution by the ODEsolve so $\bar{\mathbf{x}}_{t_{i+1}} = \text{ODEsolve}(t_{i+1} - t_i | \bar{\mathbf{x}}_{t_i}, f)$. Combining with the (2) and (9) we obtain discrete dynamics

$$\begin{cases} \bar{\mathbf{x}}_{t_{i+1}} = \text{ODEsolve}(t_{i+1} - t_i | \bar{\mathbf{x}}_{t_i}, f), \\ \mathbf{x}_{t_i} = \bar{\mathbf{x}}_{t_i} + \epsilon_{t_i}, \\ \bar{\mathbf{x}}_0 \sim F(\cdot). \end{cases} \quad (11)$$

The $\mathbf{x}(t_i)$ is observed, everything else is not. To adapt classifier 7 for this model we need to substitute intractable $\bar{\mathbf{x}}(t_i | \bar{\mathbf{x}}_0, f)$ for the best approximation based on random $\bar{\mathbf{x}}_0$ and $\mathbf{x}_{t_i}, i \in 1 \dots n$. If we take an L_2 norm in the space of random vectors, the best approximation is *conditional expectation* [41]. We denoted a series of conditional expectations by $\mathbb{E}[\bar{\mathbf{x}}(t_i) | \bar{\mathbf{x}}_0, \mathbf{x}(t_i), f]$ that means i -th series value is $\mathbb{E}[\bar{\mathbf{x}}_{t_i} | \bar{\mathbf{x}}_0, \mathbf{x}_{t_1}, \dots, \mathbf{x}_{t_n}]$. The problem of finding $\mathbb{E}[\bar{\mathbf{x}}(t_i) | \bar{\mathbf{x}}_0, \mathbf{x}(t_i), f]$ is called the *smoothing problem*. On the one hand, it is intractable in general similarly to the IVP. On the other hand, there are numerical methods for obtaining approximate solutions. Some of them are extensions of the Kalman filter [42, 43]. Another are based on the bayessian and particle filters [44].

The new classifier is defined as

$$\delta(\mathbf{x}(t_i)) = \arg \min_{j \in 1 \dots K} \rho_n(\mathbf{x}(t_i), \mathbb{E}[\bar{\mathbf{x}}(t_i) | \bar{\mathbf{x}}_0, \mathbf{x}(t_i), f_j]). \quad (12)$$

The classifier now relies not only on the dynamics driven by the vector fields f_j but also on the observations $\mathbf{x}(t_i)$. This mitigates problems discussed in the Section 3:

1. Inaccuracy from the approximate $\tilde{f}$ and ODESolve decreases because we give up the full IVP $t \in [0, T]$ in favor of the smaller IVPs on time segments $t \in [t_{i+1}, t_i]$.
2. Randomness of the $\bar{\mathbf{x}}_0, \mathbf{x}_{t_i}$ is taken into account via the conditional expectation $\mathbb{E}[\bar{\mathbf{x}}(t_i)|\bar{\mathbf{x}}_0, \mathbf{x}_{t_i}, f]$. Informally, it optimally balances the underlying dynamics (1) with the noisy observations to approximate the IVP solution $\bar{\mathbf{x}}(t_i|f)$.

Comparison of the IVP $\bar{\mathbf{x}}(t_i|\bar{\mathbf{x}}_0, f)$ approximation by $\text{ODESolve}(t_i|\bar{\mathbf{x}}_0, f)$ and by $\mathbb{E}[\bar{\mathbf{x}}(t_i)|\bar{\mathbf{x}}_0, \mathbf{x}(t_i), f]$ is illustrated on the Fig. 5.

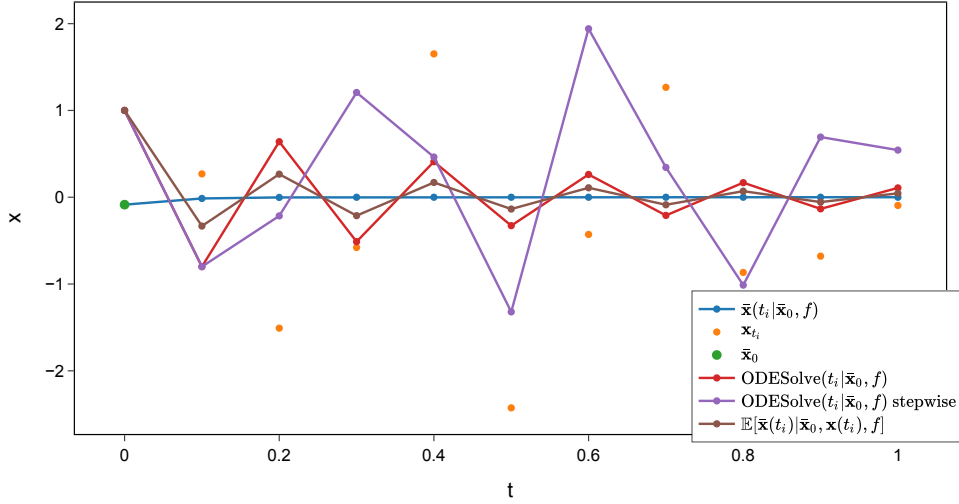


Figure 5 Different IVP approximations for the one-dimensional system $f(x) = -18x$. The initial value $x_0 \sim \mathcal{N}(1, 1)$ and $\epsilon_{t_i} \sim \mathcal{N}(0, 1)$. ODESolve uses Euler solver with initial value $\mathbb{E}[x_0] = 1$. Stepwise ODESolve uses Euler solver on consecutive time segments $[t_i, t_{i+1})$ with noisy initial values x_{t_i} . Unscented Kalman Filter (UKF) [43] is used to compute $\mathbb{E}[\bar{\mathbf{x}}(t_i)|\bar{\mathbf{x}}_0, \mathbf{x}(t_i), f]$. UKF uses stepwise Euler internally. We can see that UKF demonstrates the best IVP approximation by using both the underlying dynamics and the noisy observations. UKF is the most accurate IVP approximation with probability 0.92 by Monte-Carlo estimation.

5 Adapting classifier for SDE

We can segment dynamics (4) similarly to the (13) and obtain

$$\begin{cases} d\bar{\mathbf{x}}(t) = f(\bar{\mathbf{x}})dt + GdW(t), \\ \bar{\mathbf{x}}(0) = \bar{\mathbf{x}}_{t_i}, \\ t \in [0, t_{i+1} - t_i]. \end{cases} \quad (13)$$

If all time segments are small enough, we can approximate dynamics $f(\bar{\mathbf{x}})dt \approx \text{ODESolve}(t_{i+1} - t_i | \bar{\mathbf{x}}_{t_i}, f)$ and $GdW(t) \approx G \cdot \mathcal{N}(0, (t_{i+1} - t_i) \cdot I) = \mathcal{N}(0, (t_{i+1} - t_i) \cdot GG^T) := \xi_{t_{i+1}}$. As a result, we obtain discrete dynamics akin to (11):

$$\begin{cases} \bar{\mathbf{x}}_{t_{i+1}} = \text{ODESolve}(t_{i+1} - t_i | \bar{\mathbf{x}}_{t_i}, f) + \xi_{t_{i+1}}, \\ \mathbf{x}_{t_i} = \bar{\mathbf{x}}_{t_i} + \epsilon_{t_i}, \\ \bar{\mathbf{x}}_0 \sim F(\cdot). \end{cases} \quad (14)$$

Finding $\mathbb{E}[\bar{\mathbf{x}}(t_i) | \bar{\mathbf{x}}_0, \mathbf{x}(t_i), f]$ here is just a more general smoothing problem and can be solved by the same methods listed above. The classifier (12) for the SDE case stays the same. Comparison of the different SDE IVP $\bar{\mathbf{x}}(t_i | \bar{\mathbf{x}}_0, f)$ approximation is given on the Fig. 6.

6 Learning vector field from data

Sometimes vector field f may be unknown. Instead, time series samples from the associate dynamics (1) may be available. From the (1) and (2) we have

$$\mathbf{x}_{t_i} = \bar{\mathbf{x}}(t_i | \bar{\mathbf{x}}_0, f) + \epsilon_{t_i} \quad i \in 1 \dots n$$

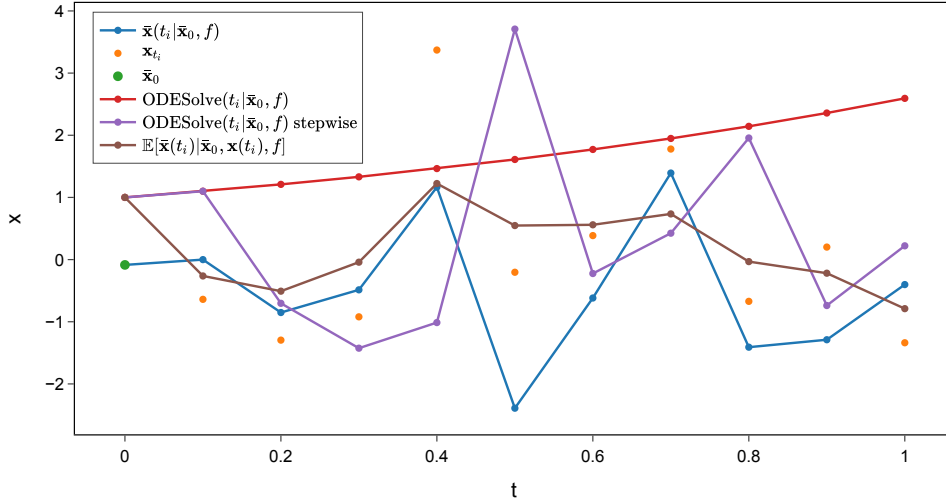


Figure 6 Different IVP approximations for the one-dimensional system $dx(t) = xdt + dW(t)$. The initial value $x_0 \sim \mathcal{N}(1, 1)$ and $\epsilon_{t_i} \sim \mathcal{N}(0, 1)$. Phase trajectory is a random process here in contrast to the ODE dynamics. ODESolve uses Euler solver with initial value $\mathbb{E}[x_0] = 1$. We can see it is inadequate model for the SDE dynamics. Stepwise ODESolve uses Euler solver on consecutive time segments $[t_i, t_{i+1})$ with noisy initial values x_{t_i} . It is more accurate but is subject to the noise and still misses $dW(t)$ term. Unscented Kalman Filter (UKF) is used to compute $\mathbb{E}[\bar{\mathbf{x}}(t_i) | \bar{\mathbf{x}}_0, \mathbf{x}(t_i), f]$. UKF uses stepwise Euler internally but also account for $dW(t)$ term. It is the most accurate IVP approximation across mentioned with probability 0.97 by Monte-Carlo estimation.

where $\bar{\mathbf{x}}_0$ is defined as in (9). Approximating the true IVP by the ODESolve we obtain

$$\mathbf{x}_{t_i} = \text{ODESolve}(t_i | \bar{\mathbf{x}}_0, f) + \epsilon_{t_i} \quad i \in 1 \dots n.$$

By taking the expectation from both sides and gathering the terms on the left-hand side we get

$$\mathbb{E}[\mathbf{x}_{t_i} - \text{ODESolve}(t_i | \bar{\mathbf{x}}_0, f)] = 0 \quad i \in 1 \dots n.$$

We can apply *generalized method of moments* (GMM) [45] here. Assume we have a set of vector fields $\mathcal{S}_\Theta \subset \mathcal{F}(M)$ parameterized by the real vector Θ . Assume $f \in \mathcal{S}_\Theta$.

Denote the solution of the following optimization problem

$$\sum_{i=1}^n (\mathbf{x}_{t_i} - \text{ODESolve}(t_i | \bar{\mathbf{x}}_0, f))^2 \rightarrow \min_{f \in \mathcal{S}_\Theta} \quad (15)$$

as $f_{\hat{\Theta}}$. Here $\bar{\mathbf{x}}_0, \mathbf{x}_{t_i}$ $i \in 1 \dots n$ are sampled values. If some technical conditions on θ holds [45], the GMM guarantees that

$$f_{\hat{\Theta}} \xrightarrow{\mathbb{P}} f, \quad n \rightarrow \infty. \quad (16)$$

The (15) can be solved using NODE. As discussed in the Section 3, n should not be large due to approximation errors of the ODESolve. Instead, it is better to sample N short batches $B = (\bar{\mathbf{x}}_0, \{\mathbf{x}_{t_i}\}_{i=1}^n)$ and solve

$$\sum_{j=1}^N \sum_{\bar{\mathbf{x}}_0, \mathbf{x}_{t_i} \in B_j} (\mathbf{x}_{t_i} - \text{ODESolve}(t_i | \bar{\mathbf{x}}_0, f))^2 \rightarrow \min_{f \in \mathcal{S}_\Theta} \quad (17)$$

which is equivalent to (15) in terms of the convergence (16).

SDE dynamics (4) can also be learned from the data by the methods similar to NODE [46, 47].

7 Empirical study

In this section we validate the proposed classifier (12) on two classification tasks of progressively decreasing prior knowledge about the underlying dynamics. In the first task the vector fields f_j are known analytically, which lets us isolate the behaviour of the smoothing-based classifier from the error of the field approximation. In the second task the fields are unknown and are learned from data with NODE as described in Section 6. The phase trajectories are still observed directly through a multivariate IMU sensor. Data is taken from the [MotionSense](#) dataset [48].

7.1 Experimental setup

Across all tasks we evaluate the classifier (12) where the conditional expectation $\mathbb{E}[\bar{\mathbf{x}}(t_i)|\bar{\mathbf{x}}_0, \mathbf{x}(t_i), f_j]$ is approximated with the Unscented Kalman Filter together with the Rauch–Tung–Striebel smoother [44]. The smoother uses an Euler solver for the ODEsolve in the (11).

Classification quality is reported as accuracy on a held-out test set. All trajectory integration and vector field learning is performed with `torchdiffeq`, the trajectory smoothing with `filterpy`.

7.2 Linear dynamical systems

The first task classifies trajectories generated by three linear ODE systems with the following form

$$f(\bar{\mathbf{x}}) = A\bar{\mathbf{x}} = U\Lambda U^T\bar{\mathbf{x}} = U \begin{pmatrix} -\gamma_1 & w & 0 \\ -w & -\gamma_1 & 0 \\ 0 & 0 & -\gamma_2 \end{pmatrix} U^T\bar{\mathbf{x}}. \quad (18)$$

Here A is a hermitian matrix with specified eigenvalue decomposition. All three systems share the same Λ but differ in eigenvector matrix U . Vector field (18) simulates damped oscillation along first two eigenvectors and exponential decay along the third. We used $\gamma_1 = 0.7, \gamma_2 = 0.6, w = 4..$ For each class we sample $\bar{\mathbf{x}}_0 \sim \mathcal{N}(\mathbf{0}, 4 \cdot I)$ and integrate the system trajectories of length $n = 100$ with step $\tau = 0.05$. Then we corrupt them with additive Gaussian noise of level $\sigma = 2$. following (2)–(3). The visualization of the final series and underlying phase trajectories is on the Fig. 7 (with lower noise level for clearer illustration).

Because the true fields f_j are available, no field learning is required. We compare our classifier against time series classification baselines: a k -Nearest-Neighbours classifier with the Dynamic Time Warping distance [2] and a recurrent (LSTM) network

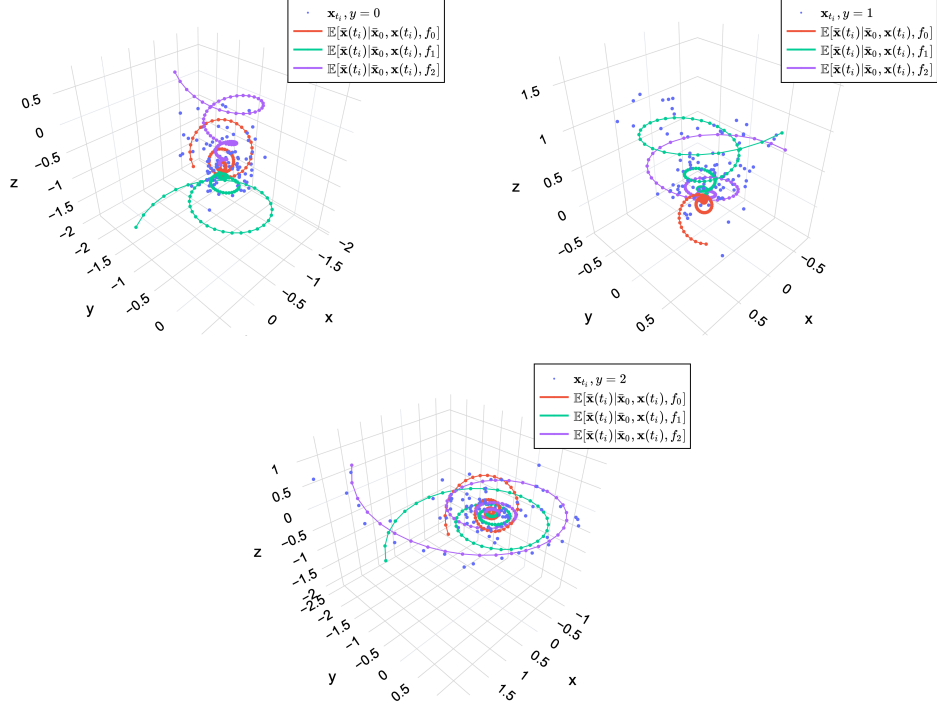


Figure 7 Observed time series $\mathbf{x}(t_i)$ generated from each linear ODE system (y label in the figure legend) and smoothed phase trajectories from every field f_0, f_1, f_2 . The systems are a combination of exponential decay and damped oscillations. Noise level for the illustration is $\sigma = 0.2$.

trained end-to-end on the raw series. Both baselines are relevant to our model. Recurrent networks have discrete dynamics inside that can be seen as an Euler solver steps [25]. DTW is used to define distance between time series [2] other than (6). Training curves for the LSTM are represented on the Fig. 8. Classification accuracy is reported in the Tab. 1.

Table 1 Classification accuracy on the linear ODE systems task ($K = 3$, $n = 100$, $\sigma = 2$).

Method	Accuracy
k -NN + DTW	0.67
LSTM	0.27
Ours (UKF smoothing)	0.86

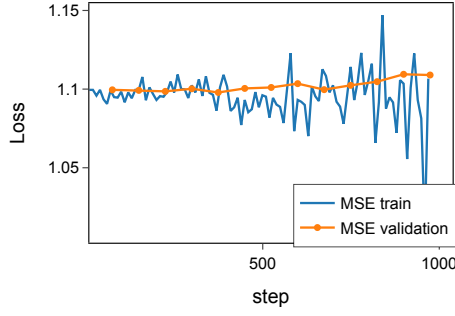


Figure 8 Learning curves for the LSTM training: MSE loss on train and validation series.

From 8 we can see that LSTM was not able to learn ODE systems. The model can't differentiate them due to the high noise level and system's "proximity" implied by the shared Λ . The k -NN + DTW classifier is a distance-based like (12) but is sub-optimal. It uses DTW to compute series distance instead of (6) and $\mathbb{E}[\bar{\mathbf{x}}(t_i)|\bar{\mathbf{x}}_0, \mathbf{x}(t_i), f]$. It also learns f_j only indirectly by memorizing training series for each class. Classifier (12) combined information from both f_j and noise level to show the best performance. Though it is not absolutely precise due to the high noise level and limited trajectory length.

7.3 Human motion recognition from IMU

The second task uses the MotionSense dataset of Inertial Measurement Unit (IMU) recordings. We took rotation rate (x, y, z) and acceleration (x, y, z) from the IMU sampled at 50 Hz as our time series. We assume them to be noisy phase trajectories following (2)–(3). We chose 6 subjects who performed 6 activities: walking, jogging, walking upstairs, walking downstairs, standing and sitting. Each activity correspond to classification class. Unlike the previous experiment, the vector fields for the activities are unknown. So for each class and each subject we learn an approximate field f_j from short trajectory batches by solving the objective (17). Subjects performed each

activity at least twice so we took one activity trial for learning and one trial for measuring accuracy. The noise level σ was estimated directly from the train data for each subject and activity.

The human body in motion is well approximated as a system of articulated rigid bodies (limbs and trunk coupled at joints), and the motion of such a system is governed by the Lagrangian equations of mechanics [49, 50]. These equations can be written in a (1) form. The IMU measures only part of the full system, but we further show that it is enough for classification. Steady locomotions (walking, jogging, stairs) are rhythmic and quasi-periodic, which in dynamical-systems language corresponds to stable limit cycles. Static postures (sitting, standing) correspond to stable equilibria. So we searched approximate fields f_j in a following form

$$f(\bar{\mathbf{x}}) = A\bar{\mathbf{x}} + \text{NN}(\bar{\mathbf{x}})$$

where A is 6×6 matrix and $\text{NN}(\cdot)$ is a small neural network. The $A\bar{\mathbf{x}}$ term corresponds to the general dynamics type (limit cycle, damped oscillations, etc.) while $\text{NN}(\bar{\mathbf{x}})$ term should account for subtle motion variations.

On the other hand, human motions can differ even within fixed activity and depend on many factors like height, age, health issues, etc. We call it an *individuality hypothesis*. Assuming the hypothesis, we learn vector fields and classify test series individually per subject. Then we aggregate classification results across subjects and report accuracy per activity and across activities. This setup does not allow to directly compare against other classification models for the MotionSense [51, 52] because they do not differentiate between subjects. We formalize the individuality hypothesis as a statistical test. Assume subjects perform an activity in a same way. Then the learned individual vector f_{subj_j} fields must be sufficiently close as well as the measured time

series. Let’s take each subject as a class and build a ”subject” classifier

$$\text{delta}(\mathbf{x}(t_i)) = \arg \min_{j \in 1 \dots \text{num subjects}} \rho_n(\mathbf{x}(t_i), \mathbb{E}[\bar{\mathbf{x}}(t_i) | \bar{\mathbf{x}}_0, \mathbf{x}(t_i), f_{\text{subj}_j}]). \quad (19)$$

If individuality hypothesis does not hold, then the classifier must be random when we apply it to the test series $\mathbf{x}(t_i)$ from all subjects. That makes our null hypothesis which we test with significance level 0.05 using binomial test [41].

Visualization of the time series and smoothed trajectories corresponding to different activities is on the Fig. ?? . We can separate smoothed trajectories for the activities that means classes have different vector fields. Accuracy is reported in the Tab. 2. High values imply ODE model truly suits human motion data. The binomial test’s p-values for the individuality hypothesis reported in the Tab. 3. We also included subject’s classification accuracy here using (19). The test failed for jogging and walking and was successful for other activities. This seems reasonable because sitting and standing are static motions with no articulated movements in contrast to jogging and walking. Rejection of the individuality hypothesis for moving up/downstairs was not expected and suggests an interesting insight.

Table 2 Activity classification accuracy on MotionSense IMU data.

Activity	Accuracy
jog	0.84
walk	0.84
upstairs	0.72
downstairs	0.70
stand	0.75
sit	0.74
all	0.765

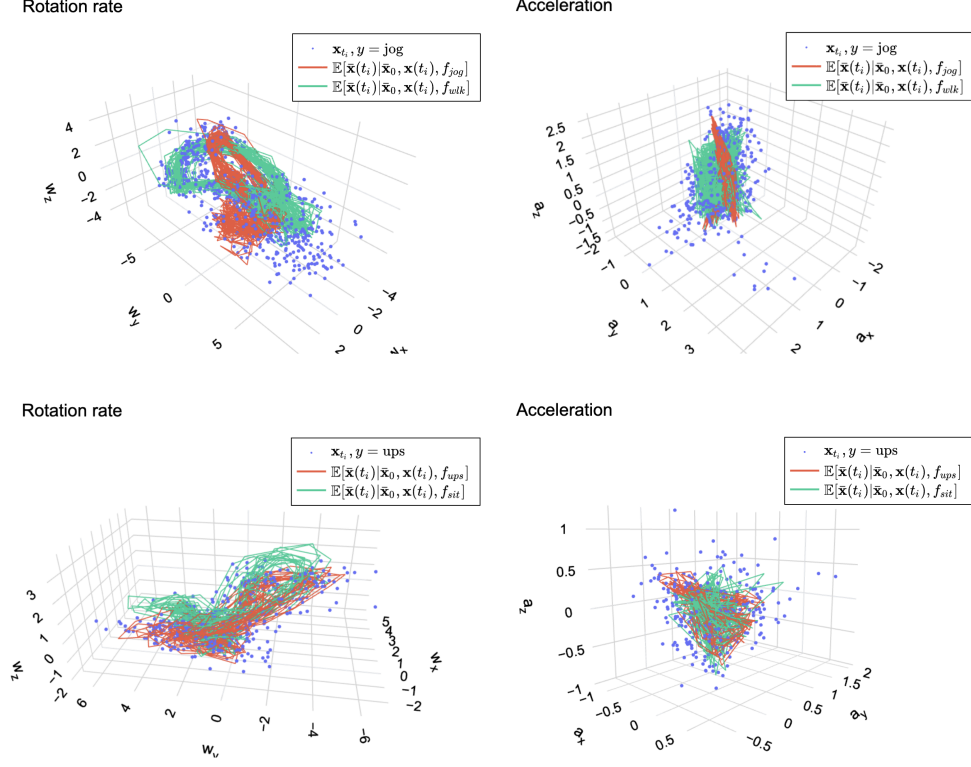


Figure 9 The upper row shows rotation rate and acceleration time series for the jogging activity. Smoothed trajectories from f_{jog} and f_{wlk} are visualized for comparison. The lower row depicts time series for upstairs activity with f_{ups} and f_{sit} smoothed trajectories. All series and fields were taken from the subject 1.

Table 3 Subject classification p-values and accuracy across activities.

Activity	p-value	Accuracy
jog	0.03	0.84
walk	0.01	1.00
upstairs	0.35	0.68
downstairs	0.34	0.71
stand	0.41	0.65
sit	0.89	0.34
all	-	0.7

8 Conclusion

In the paper we have introduced distance-based time series classifier. It assumes particular time series model — ODE or SDE dynamical systems. We proved it is

asymptotically accurate with probability one in the idealized conditions and reasonably adapted it for the more general case. Our classifier is simple and powerful as it requires only the systems' vector fields f_j . Then it uses developed framework of ODE solvers and smoothing methods. Our classifier is also flexible. If we don't know vector fields we can learn them from data. If we don't observe the entire dynamics we can use Whitney/Takens theorem to restore it. Classifier's flexibility and applicability was demonstrated empirically on the human motion signals.

Our classifier has certain limitations by its definition. Even though introduced dynamical systems are quite general they can not describe any time series. For example, we expect poor performance for non-continuous dynamics or general random processes. Next, learning vector fields may be cumbersome if we don't have some clue about their form or lack data. Finally, numerical methods the classifier uses have their own conditions and bounds that should be understood before application.

There are several directions to extend current research. First, the classifier may be improved if exact distributions for the noise and initial value are known. The theoretical classifier could be derived and empirically validated. Second, classifier may be adapted for the more general SDE dynamics. Currently, the diffusion term is $GdW(t)$ because most smoothing methods require i.i.d. diffusion samples. The general term $g(x(t), t)dW(t)$ produces dependent samples with different distributions. Lastly, the classifier may be adapted for the *change point detection* task. The change point would correspond to shifting from one dynamical system to the other. However, this requires separate investigation.

References

- [1] Berndt, D.J., Clifford, J.: Using dynamic time warping to find patterns in time series. In: Proceedings of the 3rd International Conference on Knowledge Discovery and Data Mining. AAAIWS'94, pp. 359–370. AAAI Press, ??? (1994)

- [2] Ding, H., Trajcevski, G., Scheuermann, P., Wang, X., Keogh, E.: Querying and mining of time series data: experimental comparison of representations and distance measures. *Proc. VLDB Endow.* **1**(2), 1542–1552 (2008) <https://doi.org/10.14778/1454159.1454226>
- [3] Nanopoulos, A., Alcock, R., Manolopoulos, Y.: Feature-based classification of time-series data, pp. 49–61. Nova Science Publishers, Inc., USA (2001)
- [4] Wang, X., Smith, K., Hyndman, R.: Characteristic-based clustering for time series data. *Data Min. Knowl. Discov.* **13**(3), 335–364 (2006) <https://doi.org/10.1007/s10618-005-0039-x>
- [5] Mörchen, F.: Time series feature extraction for data mining using dwt and dft (2003)
- [6] Rajpurkar, P., Hannun, A., Haghpanahi, M., Bourn, C., Ng, A.: Cardiologist-level arrhythmia detection with convolutional neural networks (2017) <https://doi.org/10.48550/arXiv.1707.01836>
- [7] Lawhern, V., Solon, A., Waytowich, N., Gordon, S., Hung, C., Lance, B.: Eegnet: a compact convolutional neural network for eeg-based brain–computer interfaces. *Journal of Neural Engineering* **15** (2018) <https://doi.org/10.1088/1741-2552/aace8c>
- [8] Malhotra, P., Ramakrishnan, A., Anand, G., Vig, L., Agarwal, P., Shroff, G.: Lstm-based encoder-decoder for multi-sensor anomaly detection (2016) <https://doi.org/10.48550/arXiv.1607.00148>
- [9] Trigeorgis, G., Ringeval, F., Brueckner, R., Marchi, E., Nicolaou, M.A., Schuller, B., Zafeiriou, S.: Adieu features? end-to-end speech emotion recognition using a deep convolutional recurrent network. In: 2016 IEEE International Conference

- on Acoustics, Speech and Signal Processing (ICASSP), pp. 5200–5204 (2016).
<https://doi.org/10.1109/ICASSP.2016.7472669>
- [10] Hammerla, N., Halloran, S., Ploetz, T.: Deep, convolutional, and recurrent models for human activity recognition using wearables. (2016)
- [11] Shi, X., Chen, Z., Wang, H., Yeung, D.-Y., Wong, W.K., WOO, W.-c.: Convolutional lstm network: A machine learning approach for precipitation nowcasting (2015)
- [12] Bifurcation of Fixed Points of Vector Fields, pp. 356–497. Springer, New York, NY (2003). https://doi.org/10.1007/0-387-21749-5_21 . https://doi.org/10.1007/0-387-21749-5_21
- [13] Øksendal, B.: Stochastic Differential Equations vol. 77, p. 326 (2000). <https://doi.org/10.2307/3619809>
- [14] Whitney, H.: Differentiable manifolds. *Annals of Mathematics* **37**(3), 645–680 (1936). Accessed 2025-11-09
- [15] Takens, F.: Detecting strange attractors in turbulence. In: *Dynamical Systems and Turbulence*, Warwick 1980: Proceedings of a Symposium Held at the University of Warwick 1979/80, pp. 366–381 (2006). Springer
- [16] Buzsáki, G., Anastassiou, C., Koch, C.: The origin of extracellular fields and currents—eeg, ecog, lfp and spikes. *Nature reviews. Neuroscience* **13**, 407–20 (2012)
<https://doi.org/10.1038/nrn3241>
- [17] Daunizeau, J., Friston, K.J., Kiebel, S.J.: Variational bayesian identification and prediction of stochastic nonlinear dynamic causal models. *Physica D: Nonlinear Phenomena* **238**(21), 2089–2118 (2009) <https://doi.org/10.1016/j.physd.2009.08.002>

- [18] Kalnay, E.: Atmospheric Modeling, Data Assimilation and Predictability. Cambridge University Press, ??? (2002)
- [19] Michelangeli, P.-A., Vautard, R., Legras, B.: Weather regimes: Recurrence and quasi stationarity. *J. Atmos. Sci.* **52**, 1237–1256 (1995) [https://doi.org/10.1175/1520-0469\(1995\)052<1237:WRRAS>2.0.CO;2](https://doi.org/10.1175/1520-0469(1995)052<1237:WRRAS>2.0.CO;2)
- [20] Wheeler, M., Hendon, H.: An all-season real-time multivariate mjo index: Development of an index for monitoring and prediction. *Monthly Weather Review - MON WEATHER REV* **132**, 1917–1932 (2004) [https://doi.org/10.1175/1520-0493\(2004\)132<1917:AARMMI>2.0.CO;2](https://doi.org/10.1175/1520-0493(2004)132<1917:AARMMI>2.0.CO;2)
- [21] Zhao, J., Li, X., Shum, C., McPhee, J.: A review of physics-based and data-driven models for real-time control of polymer electrolyte membrane fuel cells. *Energy and AI* **6**, 100114 (2021) <https://doi.org/10.1016/j.egyai.2021.100114>
- [22] Anguita, D., Ghio, A., Oneto, L., Parra, X., Reyes-Ortiz, J.L.: A public domain dataset for human activity recognition using smartphones. In: *The European Symposium on Artificial Neural Networks* (2013). <https://api.semanticscholar.org/CorpusID:6975432>
- [23] Mekruksavanich, S., Jitpattanakul, A.: Deep convolutional neural network with rnns for complex activity recognition using wrist-worn wearable sensor data. *Electronics* **10**, 1685 (2021) <https://doi.org/10.3390/electronics10141685>
- [24] Igual, R., Medrano, C., Plaza, I.: Challenges, issues and trends in fall detection systems. *Biomedical engineering online* **12**, 66 (2013) <https://doi.org/10.1186/1475-925X-12-66>
- [25] Chen, R.T.Q., Rubanova, Y., Bettencourt, J., Duvenaud, D.: Neural ordinary differential equations. In: *Proceedings of the 32nd International Conference*

- on Neural Information Processing Systems. NIPS'18, pp. 6572–6583. Curran Associates Inc., Red Hook, NY, USA (2018)
- [26] Lai, Z., Mylonas, C., Nagarajaiah, S., Chatzi, E.: Structural identification with physics-informed neural ordinary differential equations. *Journal of Sound and Vibration* **508**, 116196 (2021) <https://doi.org/10.1016/j.jsv.2021.116196>
 - [27] Lai, Z., Liu, W., Jian, X., Bacsá, K., Sun, L., Chatzi, E.: Neural Modal ODEs: Integrating Physics-based Modeling with Neural ODEs for Modeling High Dimensional Monitored Structures. <https://doi.org/10.48550/arXiv.2207.07883>
 - [28] Yildiz, C., Heinonen, M., Lahdesmaki, H.: In: *Advances in Neural Information Processing Systems*
 - [29] Du, J., Futoma, J., Doshi-Velez, F.: Model-based reinforcement learning for semi-markov decision processes with neural odes. NIPS '20. Curran Associates Inc., Red Hook, NY, USA (2020)
 - [30] Paoletti, M.E., Haut, J.M., Plaza, J., Plaza, A.: Neural ordinary differential equations for hyperspectral image classification. *IEEE Transactions on Geoscience and Remote Sensing* **58**(3), 1718–1734 (2020) <https://doi.org/10.1109/TGRS.2019.2948031>
 - [31] Ruiz-Balet, D., Zuazua, E.: Neural ode control for classification, approximation, and transport. *SIAM Review* **65**(3), 735–773 (2023) <https://doi.org/10.1137/21M1411433> <https://doi.org/10.1137/21M1411433>
 - [32] Hairer, E., Norsett, S., Wanner, G.: *Solving Ordinary Differential Equations I: Nonstiff Problems* vol. 8, (1993). <https://doi.org/10.1007/978-3-540-78862-1>
 - [33] Kingma, D.P., Welling, M.: Auto-encoding variational bayes. *CoRR*

abs/1312.6114 (2013)

- [34] Kidger, P., Morrill, J., Foster, J., Lyons, T.: Neural controlled differential equations for irregular time series. In: Proceedings of the 34th International Conference on Neural Information Processing Systems. NIPS '20. Curran Associates Inc., Red Hook, NY, USA (2020)
- [35] Herrera, C., Krach, F., Teichmann, J.: Neural jump ordinary differential equations: Consistent continuous-time prediction and filtering. arXiv: Machine Learning (2020)
- [36] Seedat, N., Imrie, F., Bellot, A., Qian, Z., Schaar, M.: Continuous-time modeling of counterfactual outcomes using neural controlled differential equations. In: International Conference on Machine Learning (2022). <https://api.semanticscholar.org/CorpusID:249712326>
- [37] Ouala, S., Fablet, R., Drumetz, L., Chapron, B., Pascual, A., Collard, F., Gaultier, L.: Physically informed neural networks for the simulation and data-assimilation of geophysical dynamics, pp. 3490–3493 (2020). <https://doi.org/10.1109/IGARSS39084.2020.9323081>
- [38] Ouala, S., Nguyen, D., Drumetz, L., Chapron, B., Pascual, A., Collard, F., Gaultier, L., Fablet, R.: Learning latent dynamics for partially observed chaotic systems. *Chaos: An Interdisciplinary Journal of Nonlinear Science* **30**, 103121 (2020) <https://doi.org/10.1063/5.0019309>
- [39] Arnold, V.I.: Ordinary Differential Equations. Springer, Berlin, Heidelberg (1992). <https://doi.org/10.1007/978-3-642-58088-8>
- [40] Hairer, E., Wanner, G.: Solving Ordinary Differential Equations II. Stiff and Differential-Algebraic Problems vol. 14, (1996). <https://doi.org/10.1007/>

- [41] Hogg, R.V., McKean, J.W., Craig, A.T.: Introduction to Mathematical Statistics. Pearson education international. Pearson Education, ??? (2005)
- [42] Ribeiro, M., Ribeiro, I.: Kalman and Extended Kalman Filters: Concept, Derivation and Properties
- [43] Julier, S.J., Uhlmann, J.K.: New extension of the kalman filter to nonlinear systems. In: Defense, Security, and Sensing (1997). <https://api.semanticscholar.org/CorpusID:7937456>
- [44] Särkkä, S.: Bayesian Filtering and Smoothing, (2013). <https://doi.org/10.1017/CBO9781139344203>
- [45] Hansen, L.P.: Large sample properties of generalized method of moments estimators. *Econometrica* **50**(4), 1029–1054 (1982)
- [46] Li, X., Wong, T.-K.L., Chen, R.T.Q., Duvenaud, D.: Scalable gradients for stochastic differential equations. In: Chiappa, S., Calandra, R. (eds.) Proceedings of the Twenty Third International Conference on Artificial Intelligence and Statistics. Proceedings of Machine Learning Research, vol. 108, pp. 3870–3882. PMLR, ??? (2020). <https://proceedings.mlr.press/v108/li20i.html>
- [47] Kidger, P., Foster, J., Li, X.C., Lyons, T.: Efficient and accurate gradients for neural sdes. In: Ranzato, M., Beygelzimer, A., Dauphin, Y., Liang, P.S., Vaughan, J.W. (eds.) Advances in Neural Information Processing Systems, vol. 34, pp. 18747–18761. Curran Associates, Inc., ??? (2021). https://proceedings.neurips.cc/paper_files/paper/2021/file/9ba196c7a6e89eafd0954de80fc1b224-Paper.pdf

- [48] Malekzadeh, M., Clegg, R.G., Cavallaro, A., Haddadi, H.: Mobile sensor data anonymization. In: Proceedings of the International Conference on Internet of Things Design and Implementation. IoTDI '19, pp. 49–58. ACM, New York, NY, USA (2019). <https://doi.org/10.1145/3302505.3310068> . <http://doi.acm.org/10.1145/3302505.3310068>
- [49] Winter, D.A.: Biomechanics and Motor Control of Human Movement, 4th edn. John Wiley & Sons, Hoboken, New Jersey (2009)
- [50] McGeer, T.: Passive dynamic walking. I. J. Robotic Res. **9**, 62–82 (1990) <https://doi.org/10.1177/027836499000900206>
- [51] Saha, U., Saha, S., Kabir, M.T., Fattah, S.A., Saquib, M.: Decoding human activities: Analyzing wearable accelerometer and gyroscope data for activity recognition. IEEE Sensors Letters **8**(8), 1–4 (2024) <https://doi.org/10.1109/LSENS.2024.3423340>
- [52] Bodhe, R., Sivakumar, S., Sakarkar, G., Juwono, F., Apriono, C.: Outdoor activity classification using smartphone based inertial sensor measurements. Multimedia Tools and Applications **83**, 1–27 (2024) <https://doi.org/10.1007/s11042-024-18599-w>

Appendix A Proofs and examples

Proof of Theorem 1. First, estimate conditional probability of the misclassification

$$\begin{aligned} \mathbb{P}(\delta(\mathbf{x}(t_i)) = j | y = p) &= \mathbb{P}\left[\bigcap_{s=1, s \neq j}^K \{\rho_n(\mathbf{x}(t_i), \bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, f_j)) < \rho_n(\mathbf{x}(t_i), \bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, f_s))\}\right] \leq \\ &| \text{monotonicity of probability} | \leq \mathbb{P}[\rho_n(\mathbf{x}(t_i), \bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, f_j)) < \rho_n(\mathbf{x}(t_i), \bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, f_p))]. \end{aligned}$$

Expand metrics in the last event using $y = p$:

$$\rho_n(\mathbf{x}(t_i), \bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, f_p)) = \sum_{i=1}^n \|\mathbf{x}(t_i) - \bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, f_p)\|^2 = \sum_{i=1}^n \|\epsilon_{t_i}\|^2,$$

$$\begin{aligned} \rho_n(\mathbf{x}(t_i), \bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, f_j)) &= \sum_{i=1}^n \|\mathbf{x}(t_i) - \bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, f_j)\|^2 = \sum_{i=1}^n \|(\bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, f_p) - \bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, f_j)) + \epsilon_{t_i}\|^2 = \\ &= \sum_{i=1}^n \|\bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, f_p) - \bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, f_j)\|^2 + 2 \sum_{i=1}^n \langle \bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, f_p) - \bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, f_j), \epsilon_{t_i} \rangle + \sum_{i=1}^n \|\epsilon_{t_i}\|^2. \end{aligned}$$

Here $\langle \cdot, \cdot \rangle$ is the standard scalar product. The last event is now

$$\begin{aligned} \rho_n(\mathbf{x}(t_i), \bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, f_i)) &< \rho_n(\mathbf{x}(t_i), \bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, f_j)) \Leftrightarrow \\ \sum_{i=1}^n \langle \bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, f_p) - \bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, f_j), \epsilon_{t_i} \rangle &< -\frac{1}{2} \sum_{i=1}^n \|\bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, f_p) - \bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, f_j)\|^2 \Leftrightarrow \\ \sum_{i=1}^n \langle \bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, f_p) - \bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, f_j), \epsilon_{t_i} \rangle &< -\frac{1}{2} \rho_n(\bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, f_j), \bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, f_p)). \end{aligned}$$

Apply Markov inequality and use (3):

$$\begin{aligned} \mathbb{P}[\rho_n(\mathbf{x}(t_i), \bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, f_j)) < \rho_n(\mathbf{x}(t_i), \bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, f_p))] &\leq 4 \frac{\mathbb{D}[\sum_{i=1}^n \langle \bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, f_p) - \bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, f_j), \epsilon_{t_i} \rangle]}{\rho_n^2(\bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, f_j), \bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, f_p))} = \\ &= 4 \frac{\sum_{i=1}^n \mathbb{E}[\langle \bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, f_p) - \bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, f_j), \epsilon_{t_i} \rangle^2]}{\rho_n^2(\bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, f_j), \bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, f_p))} = \\ &= 4 \frac{\sum_{i=1}^n (\bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, f_p) - \bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, f_j))^T \mathbb{E}[\epsilon_{t_i} \epsilon_{t_i}^T] (\bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, f_p) - \bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, f_j))}{\rho_n^2(\bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, f_j), \bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, f_p))} = \\ &= 4 \frac{\sum_{i=1}^n (\bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, f_p) - \bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, f_j))^T R (\bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, f_p) - \bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, f_j))}{\rho_n^2(\bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, f_j), \bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, f_p))} = \end{aligned}$$

$$\begin{aligned}
&= 4 \frac{\sum_{i=1}^n \langle \bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, f_p) - \bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, f_j) \rangle^T, R(\bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, f_p) - \bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, f_j))}{\rho_n^2(\bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, f_j), \bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, f_p))} \leq \\
&\leq 4\sigma^2 \frac{\sum_{i=1}^n \|\bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, f_p) - \bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, f_j)\|^2}{\rho_n^2(\bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, f_j), \bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, f_p))} = 4\sigma^2 \frac{\rho_n(\bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, f_j), \bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, f_p))}{\rho_n^2(\bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, f_j), \bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, f_p))} = \frac{4\sigma^2}{\rho_n(\bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, f_j), \bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, f_p))} := \xi_n.
\end{aligned}$$

By theorem condition we have

$$\rho_n(\bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, f_j), \bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, f_p)) \rightarrow \rho_\infty(\bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, f_j), \bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, f_p)) = +\infty, \quad n \rightarrow \infty$$

so $\xi_n \rightarrow 0$, $n \rightarrow \infty$ for all $j, p \in 1 \dots K$.

Second, rewrite probability of the misclassification via conditional probability:

$$\begin{aligned}
\mathbb{P}(\delta(\mathbf{x}(t_i)) \neq y) &= \mathbb{E}_y \left[\sum_{p=1}^K \mathbb{P}(\delta(\mathbf{x}(t_i)) \neq p | y = p) \right] = \mathbb{E}_y \left[\sum_{p=1}^K \mathbb{P} \left(\bigcup_{j=1, j \neq p}^K \{\delta(\mathbf{x}(t_i)) = j | y = p\} \right) \right] \leq \\
| \text{union bound} | &\leq \mathbb{E}_y \left[\sum_{p=1}^K \sum_{j=1, j \neq p}^K \mathbb{P}(\delta(\mathbf{x}(t_i)) = j | y = p) \right] \leq \sum_{p=1}^K \sum_{j=1, j \neq p}^K \xi_n = K(K-1)\xi_n \rightarrow 0.
\end{aligned}$$

□

Example 1. Here we construct a system of two arbitrary close vector fields with diverging phase trajectories. Let $M = \{\bar{\mathbf{x}} \in \mathbb{R}^2 \mid \|\bar{\mathbf{x}}\|_2 \leq R\}$ be a plane circle. Define the vector field's norm as a sup norm

$$\|f\| = \sup_{\bar{\mathbf{x}} \in M} \|f(\bar{\mathbf{x}})\|_2.$$

Let $\bar{\mathbf{x}}_0 = c \cdot (\cos \gamma, \sin \gamma)^T \neq \mathbf{0}$ be an initial value. Let $\{t_i\}_{i=1}^\infty$ be any time sequence and let $r > 0$ be an arbitrary small value. Construct two vector fields

$$f(\bar{\mathbf{x}}) = \begin{pmatrix} ia & 0 \\ 0 & ia \end{pmatrix} \bar{\mathbf{x}} = A\bar{\mathbf{x}},$$

$$\tilde{f}(\bar{\mathbf{x}}) = \begin{pmatrix} ia - b & 0 \\ 0 & ia - b \end{pmatrix} \bar{\mathbf{x}} = \tilde{A}\bar{\mathbf{x}}$$

where i is an imaginary unit, $a \neq 0, b > 0$ are some scalars. Then

$$\|f - \tilde{f}\| \leq \|A - \tilde{A}\|_2 \cdot \|x\| = \left\| \begin{pmatrix} b & 0 \\ 0 & b \end{pmatrix} \right\|_2 \cdot \|x\| \leq bR$$

where $\|A - \tilde{A}\|_2$ is matrix spectral norm. So if $b < \frac{r}{R}$ then $\|f - \tilde{f}\| < r$. Next, it easy to obtain that IVP solutions for the ODE systems are

$$\bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, f) = c \begin{pmatrix} \sin(\gamma - at) \\ \cos(\gamma - at) \end{pmatrix},$$

$$\bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, \tilde{f}) = c \exp(-bt) \begin{pmatrix} \sin(\gamma - at) \\ \cos(\gamma - at) \end{pmatrix}.$$

It is obvious that $\bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, \tilde{f}) \rightarrow \mathbf{0}, t \rightarrow \infty$ and $\|\bar{\mathbf{x}}(t|\bar{\mathbf{x}}_0, \tilde{f})\| = \|\bar{\mathbf{x}}_0\| = c \forall t$. So

$$\|\bar{\mathbf{x}}(t_i|\bar{\mathbf{x}}_0, f) - \bar{\mathbf{x}}(t_i|\bar{\mathbf{x}}_0, \tilde{f})\| \rightarrow c \neq 0, i \rightarrow \infty$$

and then

$$\rho_n(\bar{\mathbf{x}}(t_i|\bar{\mathbf{x}}_0, f), \bar{\mathbf{x}}(t_i|\bar{\mathbf{x}}_0, \tilde{f})) = \sum_{i=1}^n \|\bar{\mathbf{x}}(t_i|\bar{\mathbf{x}}_0, f) - \bar{\mathbf{x}}(t_i|\bar{\mathbf{x}}_0, \tilde{f})\|_2^2 \rightarrow \infty, \quad n \rightarrow \infty$$

by the absence of the necessary condition for series convergence.

As a result, classifier (7) that uses $\bar{\mathbf{x}}(t_i|\bar{\mathbf{x}}_0, \tilde{f})$ do not have guarantees of the Theorem 1 even though f and $\tilde{f}$ are arbitrary close.